The Campus of Tomorrow

Higher Colleges of Technology

كليات التقنية العليا

CDS 2413- PROGRAMMING FOR DATA SCIENCE

Week 9-10

Week 9	CLO 3: Create scripts to solve business problems using data analysis techniques such as classification, clustering, optimization, and recommendation

Document Revision CONTROL (DRC)

Version	Author	Effective Date	Change Description	DRC No
1.0	Mohammed Abdul Rahim	2/05/2023	Define the first version	001

X.Y
X: Major change
Y: minor change

Lecture Outline

- Classification and Model Selection
- Logistic Regression
- K-Nearest Neighbours-
- Naïve Bayes
- Decision Tree
- Python Programming for Classification

LECTURE OBJECTIVES

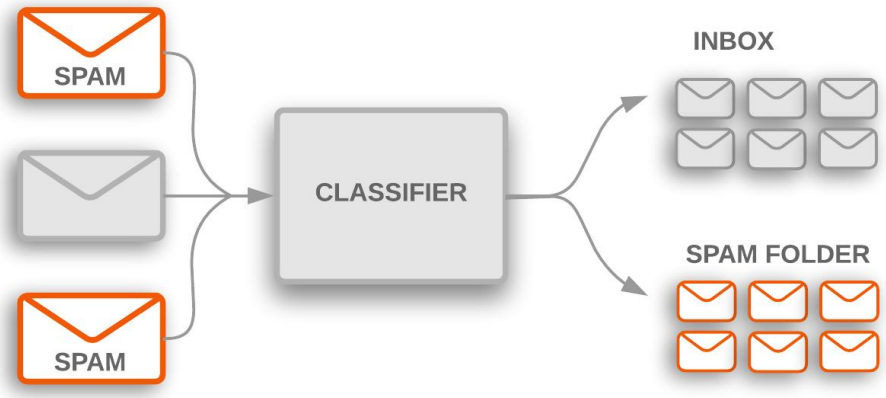
- The student should be able to do the following at the end of the lecture:
  - Understand the **basic concepts** of classification models.
  - Implement **classification models** using Python.
  - Interpret the **results** of classification models.
  - Apply **classification models** to real-world problems.

Classification

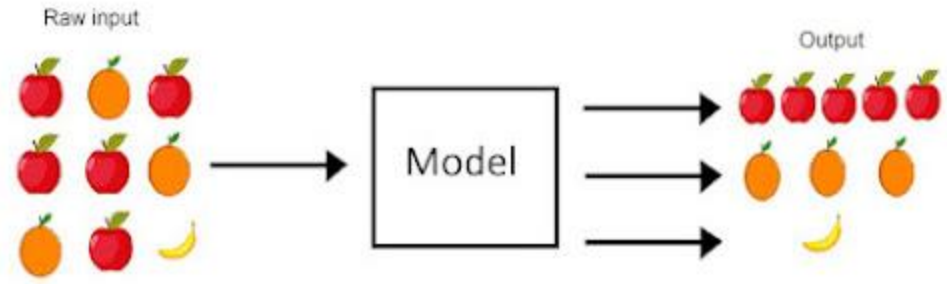
What is Classification?

- **Classification** is a technique that separates data points into different **classes**.
- It organizes **datasets** of all sizes and complexities.
- The algorithm used for classification is called a **classifier**.
- Observations made through classification are called **instances**.

What is Classification?



Email classified into inbox emails and spam emails



In image segmentation, classifiers are used to classify the images

What is Classification?

- Classification models are used in a wide variety of applications, including:
  - Spam filtering
  - Fraud detection
  - Medical diagnosis
  - Customer segmentation
  - Image classification
  - Text classification

- The most common classification algorithms include:
  - Logistic regression
  - Naïve Bayes
  - Decision Tree
  - K-nearest neighbors

Classification – Model Selection

Classification - Confusion Matrix

- The **confusion matrix** is used to evaluate the performance of a classification model.
- It provides a detailed breakdown of
  - **True Positives (TP)**,
  - **True Negatives (TN)**,
  - **False Positives (FP)**, and
  - **False Negatives (FN)**.

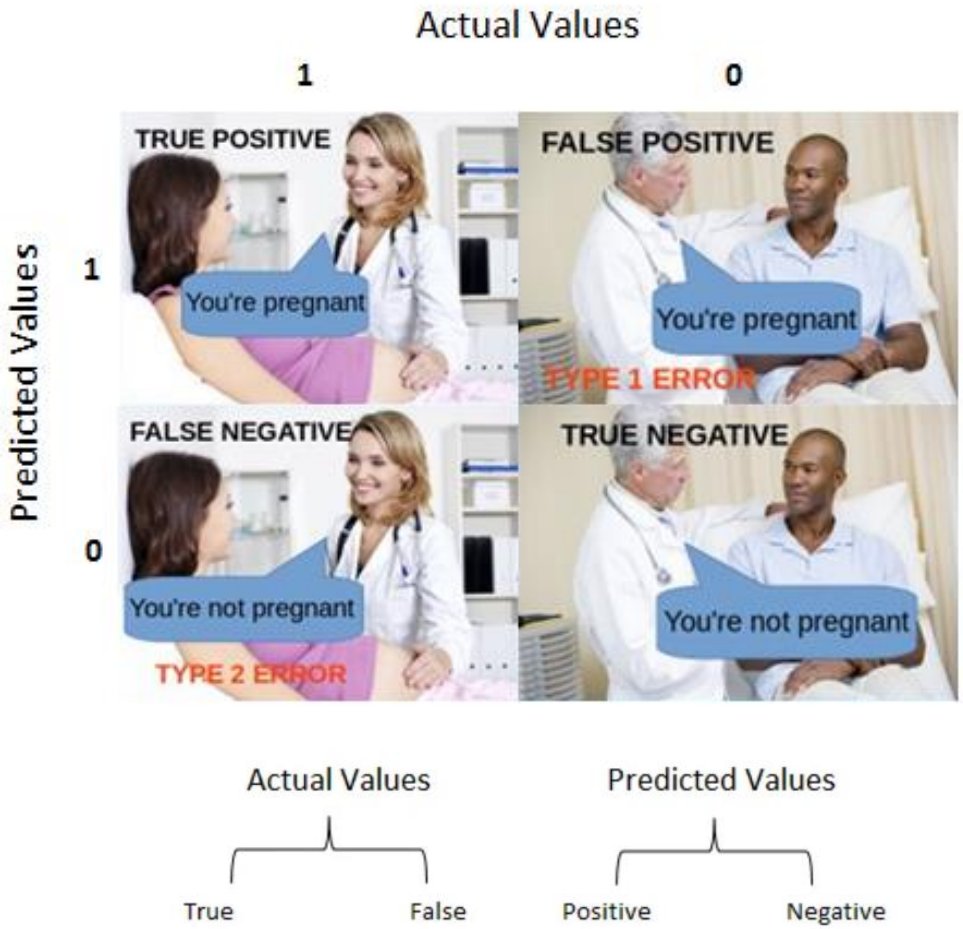
Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

https://towardsdatascience.com/understanding-the-confusion-matrix-from-scikit-learn-c51d88929c79

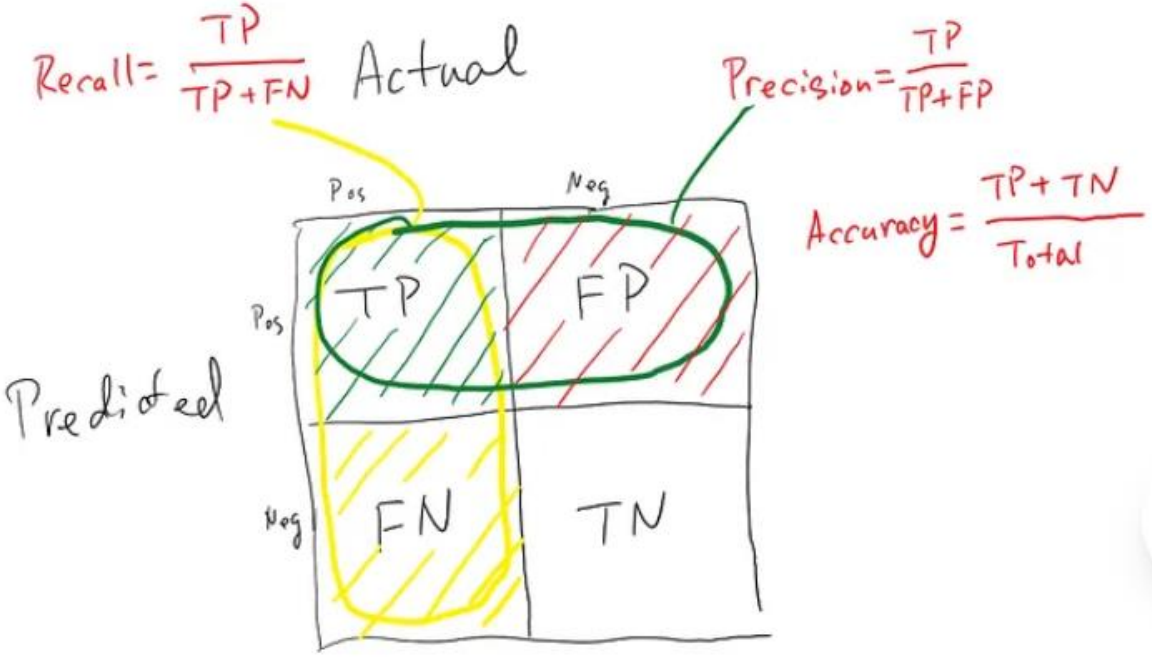
Classification - Confusion Matrix

- Checking whether a patient have pregnancy (Actual – Red font, Predicted-Green font)
- **True Positive (TP):** Predicting a pregnant woman as pregnant.
- **True Negative (TN):** Predicting a non-pregnant man as non-pregnant.
- **False Positive (FP) - Type I Error :** Predicting a non-pregnant man as pregnant.
- **False Negative (FN) - Type 2 Error :** Predicting a pregnant woman as non-pregnant.



Classification - Confusion Matrix

- **Accuracy**: The ratio of correctly predicted instances to the total instances.
- **Precision**: The ratio of correctly predicted positive observations to the total predicted positives.
- **Recall**: The ratio of correctly predicted positive observations to all observations in the actual class.
- **Error Rate**: The ratio of incorrect predictions (FP + FN) to the total instances.

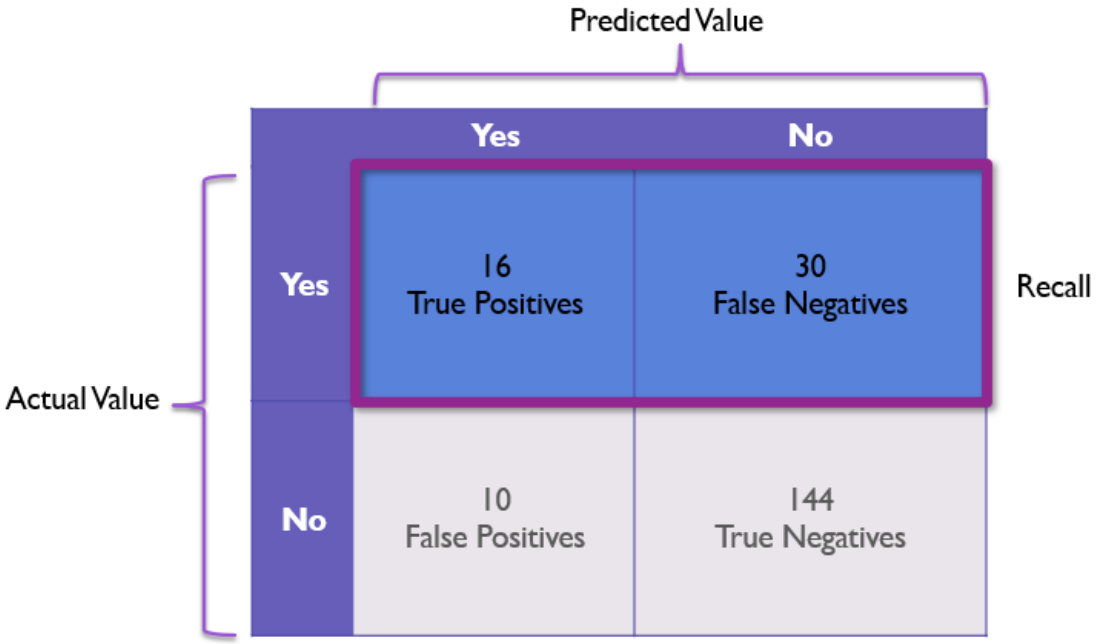


$$\text{Error} = \frac{FP + FN}{\text{Total}}$$

Understanding how to calculate accuracy, precision, recall, and error rate from a confusion matrix is essential for evaluating the performance of classification models.

Classification - Confusion Matrix

- Evaluate the accuracy rate, recall, error and precision of the given model as show in the image:



1. Classification – Logistic Regression

Classification – Logistic Regression

- Predicts a **categorical dependent variable** from independent variables.
- **Example:** Will you purchase health insurance? (Yes/No)
  - **Dependent Variable**: **Health Insurance Purchase**
  - **Independent Variables**: **Age**, **Income**, etc.
- **Logistic Regression Equation**:
  - **Log-odds** of the dependent variable is modeled as a linear combination of the independent variables.

Dependent variable

Independent variable

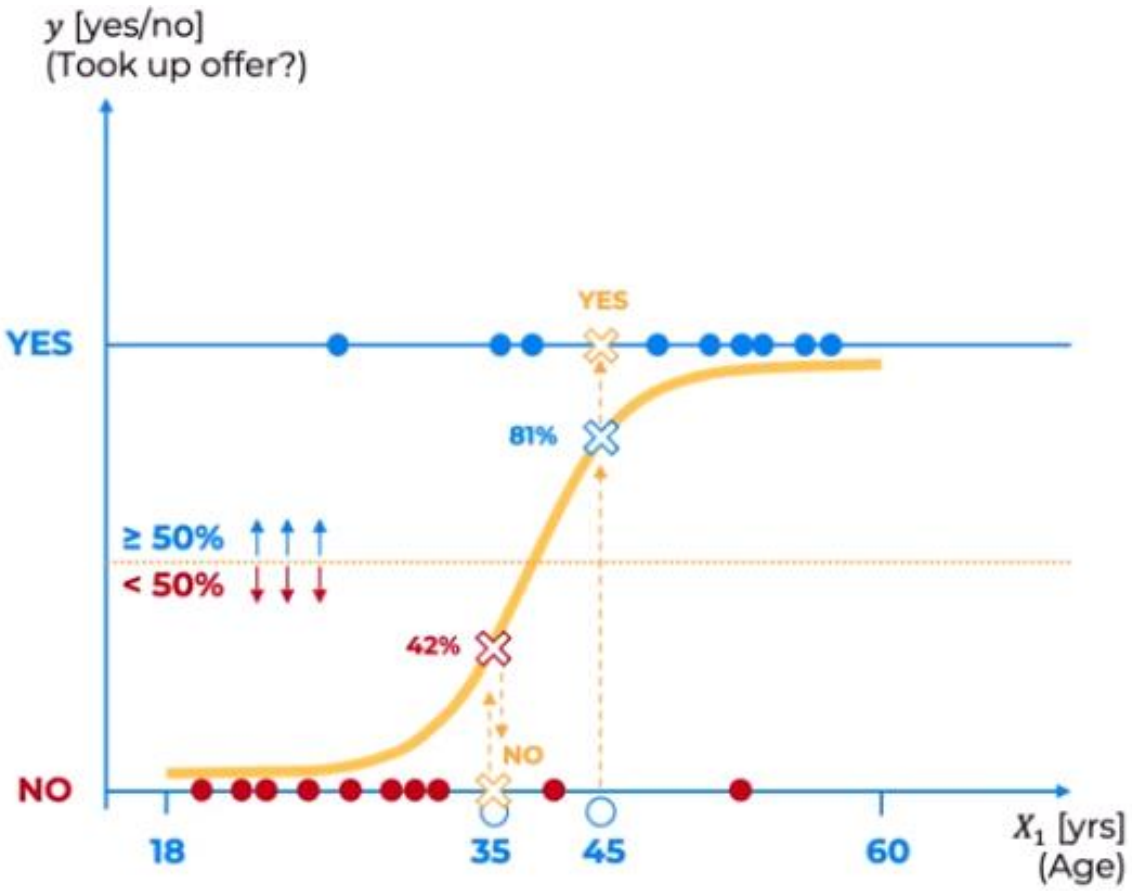
Based on

Age

Will you purchase health insurance? Yes/No → Categorical variable

Logistic Regression Equation $\ln \frac{p}{1-p} = b_0 + b_1 X_1$

Classification – Logistic Regression



There is a chance of 42% of age 35 to get the insurance and 81% chance for age 45

Classification – Logistic Regression

Dependent variable

Will you purchase health insurance? Yes/No → Categorical variable

Based on

Independent variables

Age

Income

Level of Education

Family or Single

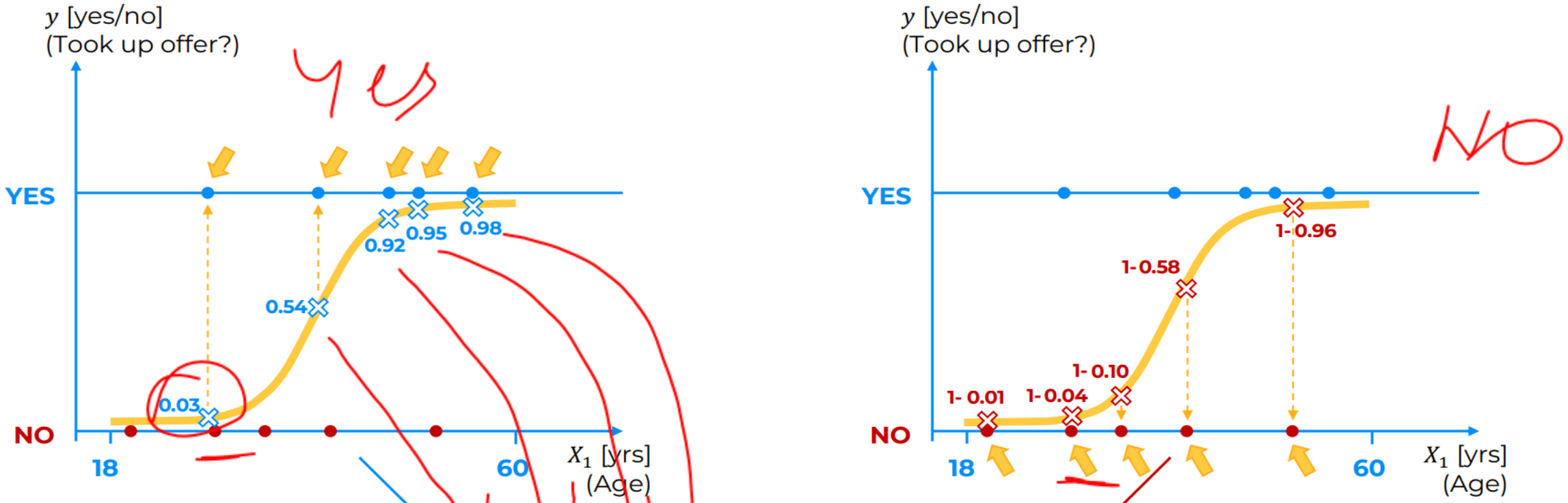
Logistic Regression Equation

$$\ln \frac{p}{1-p} = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4$$

Classification – Logistic Regression

- **Maximum Likelihood**
  - Maximum likelihood classification assumes that the statistics for each class in each band are normally distributed and
  - Calculates the probability that a given pixel belongs to a specific class.
  - Unless you select a probability threshold, all pixels are classified.

Classification – Logistic Regression

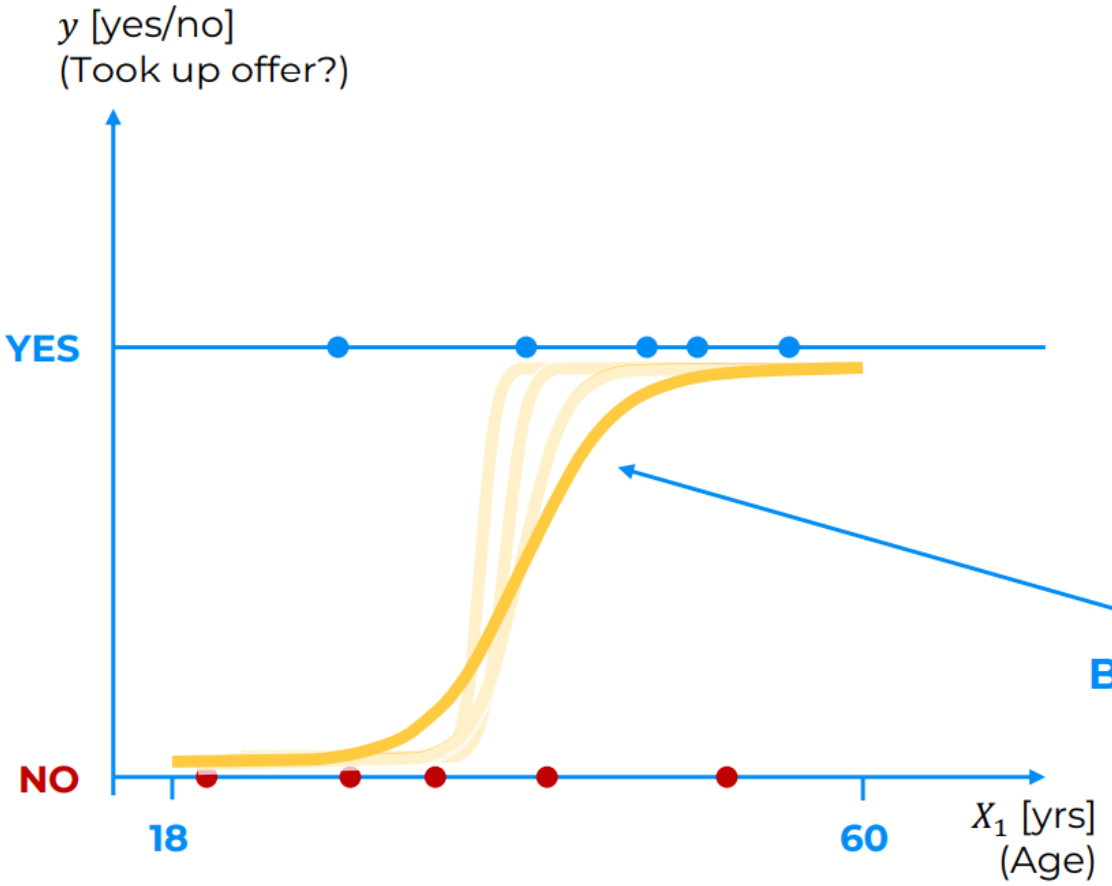


Likelihood = 0.03 x 0.54 x 0.92 x 0.95 x 0.98 x (1 – 0.01) x (1 – 0.04) x (1 – 0.10) x (1 – 0.58) x (1 – 0.96)

Probability of saying Yes at different ages

Probability of saying No at different ages

Classification – Logistic Regression



Likelihood = 0.00007418

Likelihood = 0.00012845

Likelihood = 0.00016553

Likelihood = 0.00019939

Best Curve <= Maximum Likelihood

https://sds-platform-private.s3-us-east-2.amazonaws.com/uploads/Machine-Learning-Level-1-Downloadable-Slides-v1.4.pdf

Classification – Logistic Regression - Python

Outline

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression


# Load the data
data = pd.read_csv('data.csv')

# Separate the independent and dependent variables
X = data[['independent_variable1', 'independent_variable2']]
y = data['dependent_variable']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Create and fit the logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```

Classification – Logistic Regression - Python

Outline

```
# Make predictions on the test set
predictions = model.predict(X_test)

# Evaluate the model's performance
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
```

Build, Train, Predict and Evaluate the Logistic Regression Model– Case Study

Classification- Python– Dataset

- For developing classification models based on different algorithm, this course use dataset called as BreastCancer.CSV
- Dataset is used to find the class of breast cancer (dependent variable) based on the ten independent variables
  - Class=2(Tumor)
  - Class=4(Malignant)

Independent variables

Dependent variable

Sample_Id	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	Bland Chromatin	Normal Nucleoli	Mitoses	Class
1000025	5	1	1	1	2	1	3	1	1	2
1002945	5	4	4	5	7	10	3	2	1	2
1015425	3	1	1	1	2	2	3	1	1	2
1016277	6	8	8	1	3	4	3	7	1	2
1017023	4	1	1	3	2	1	3	1	1	2
1017122	8	10	10	8	7	10	9	7	1	4
1018099	1	1	1	1	2	10	3	1	1	2
1018561	2	1	2	1	2	1	3	1	1	2
1033078	2	1	1	1	2	1	1	1	5	2

Classification – Logistic Regression - Python

Steps

1. **Load Dataset**: BreastCancer.csv
2. **Preprocess Data**: Handle missing values, encode categorical variables.
3. **Split Data**: Train and Test sets.
4. **Build Model**: Using LogisticRegression from sklearn.
5. **Train Model**: Fit the model on the training set.
6. **Predict**: Make predictions on the test set.
7. **Evaluate**: Using confusion matrix and accuracy metrics

Classification – Logistic Regression - Python

Importing the libraries

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Classification – Logistic Regression - Python

Importing the dataset

```
dataset = pd.read_csv('BreastCancer.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
print(X)
print(y)
```

All columns except the last column

Independent variables

Dependent variable

All rows

Last column

Sample_Id	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	Bland Chromatin	Normal Nucleoli	Mitoses	Class
1000025	5	1	1	1	2	1	3	1	1	2
1002945	5	4	4	5	7	10	3	2	1	2
1015425	3	1	1	1	2	2	3	1	1	2
1016277	6	8	8	1	3	4	3	7	1	2
1017023	4	1	1	3	2	1	3	1	1	2
1017122	8	10	10	8	7	10	9	7	1	4
1018099	1	1	1	1	2	10	3	1	1	2
1018561	2	1	2	1	2	1	3	1	1	2
1033078	2	1	1	1	2	1	1	1	5	2

X

```
[[1000025       5       1 ...       3       1       1]
 [1002945       5       4 ...       3       2       1]
 [1015425       3       1 ...       3       1       1]
```

y

```
2 2 2 2 4 2 2 2 2 2 2 4 2 4 4 2 2 4 2 4 4 2 2 4 2 2 2 2 2 2 4 2 2 2 4 2
```

Classification – Logistic Regression - Python

Splitting the dataset into the Training set and Test set

```
[33] from sklearn.model_selection import train_test_split
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
```

```
print(X_test)
```

```
[[1173347       1       1 ...       1       1       1]
 [1156017       3       1 ...       2       1       1]
 [ 706426       5       5 ...       4       3       1]
```

```
print(y_test)
```

```
[2 2 4 4 2 2 2 4 2 2 4 2 4 2 2 2 4 4 4 2 2 2 4 2 4 4 2 2 2 4 2 4 4 2 2 2 4
 4 2 4 2 2 2 2 2 2 2 4 2 2 4 2 4 2 2 2 4 2 2 4 2 2 2 2 2 2 2 2 4 4 2 2 2 2
 2 2 4 2 2 2 4 2 4 2 2 4 2 2 4 2 4 2 4 4 4 2 4 4 4 2 2 2 4 4 2 2 4 4 2 2 4
 2 2 4 2 2 2 4 2 2 2 4 2 2 4 4 2 4 2 4 2 2 4 2 2 4 2 4 2 2 2 4 2 2 2 2 4 2
 4 2 4 4 2 2 2 2 4 4 2 4 4 4 4 4 4 2 2 2 2 2 2]
```

Classification – Logistic Regression - Python

Feature Scaling

```
[10] from sklearn.preprocessing import StandardScaler
     sc = StandardScaler()
     X_train = sc.fit_transform(X_train)
     X_test = sc.transform(X_test)
```

Classification – Logistic Regression - Python

Training the Logistic Regression model on the Training set

```
[11] from sklearn.linear_model import LogisticRegression
     classifier = LogisticRegression(random_state = 0)
     classifier.fit(X_train, y_train)
```

Configuring for logistic regression model by mentioning the classifier is logistic regression

Attaching the classifier to the train dataset

Classification – Logistic Regression - Python

Making the Confusion Matrix

```
[35] from sklearn.metrics import confusion_matrix, accuracy_score
     y_pred = classifier.predict(X_test)
     cm = confusion_matrix(y_test, y_pred)
     print(cm)
     accuracy_score(y_test, y_pred)
```

```
[[107   0]
 [ 64   0]]
0.6257309941520468
```

False Positive(FP)

False Negative(FN)

Accuracy Rate

Number of Errors = FP + FN = 0+ 64 = 64

Classification – Logistic Regression - Python

Predicting the value

```
[38] print(classifier.predict(sc.transform([[1173347,1,1,1,1,2,5,1,1,1]])))
```

```
[2]
```

It predicted correctly that class=2. So, the model is working fine.

```
print(X_test)
```

```
[[1173347       1       1 ...       1       1       1]
 [1156017       3       1 ...       2       1       1]
 [ 706426       5       5 ...       4       3       1]
```

```
print(y_test)
```

```
[2 2 4 4 2 2 2 4 2 2 4 2 4 2 2 2 4 4 4 2 2 2 4 2 4 4 2 2 2 4 2 4 4 2 2 2 4
 4 2 4 2 2 2 2 2 2 2 4 2 2 4 2 4 2 2 2 4 2 2 4 2 2 2 2 2 2 2 2 4 4 2 2 2 2
 2 2 4 2 2 2 4 2 4 2 2 4 2 2 4 2 4 2 4 4 4 2 4 4 4 2 2 2 4 4 2 2 4 4 2 2 4
 2 2 4 2 2 2 4 2 2 2 4 2 2 4 4 2 4 2 4 2 2 4 2 2 4 2 4 2 2 2 4 2 2 2 2 4 2
 4 2 4 4 2 2 2 2 4 4 2 4 4 4 4 4 4 2 2 2 2 2 2]
```

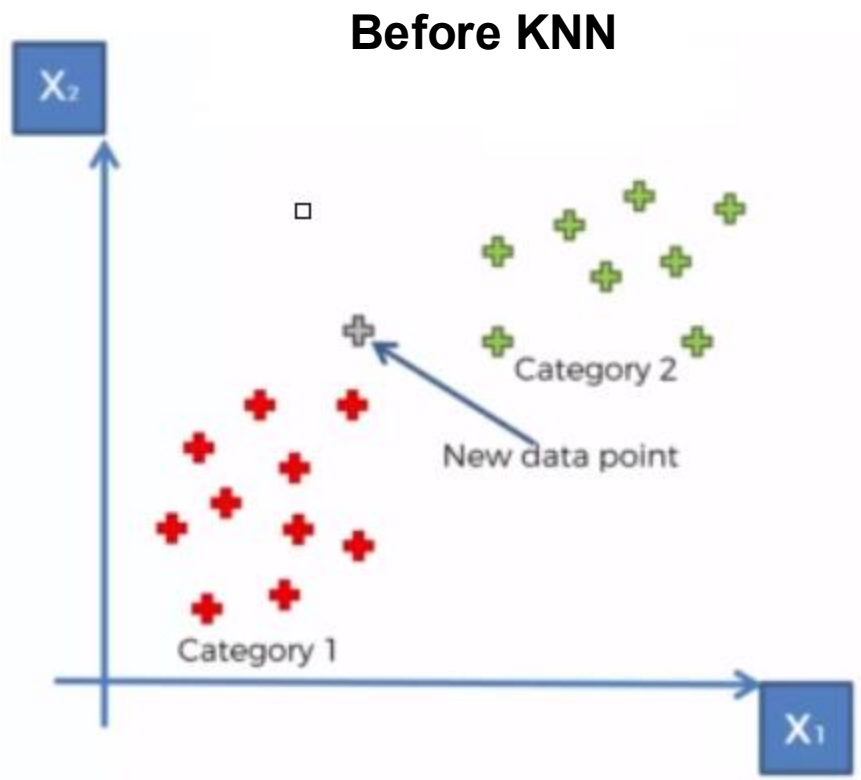
Check in BreastCancer.csv file if data is not fully visible

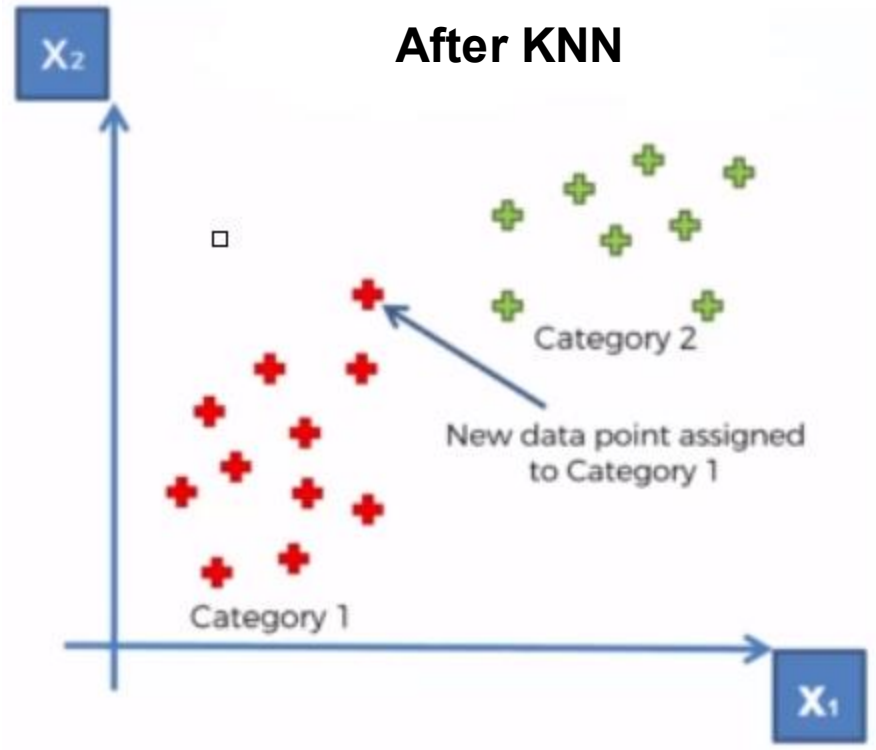
2. Classification – K-Nearest Neighbor (KNN)

Classification - KNN

- K-nearest neighbors (KNN) algorithm is used for both **classification** and **regression**.
- It works by finding the **k most similar** points to a new data point.
- Steps:
  - **Calculate distance** between the new data point and all training examples.
  - **Identify the k nearest neighbors**.
  - **Predict the class** based on the majority class of the k neighbors.

Classification – KNN



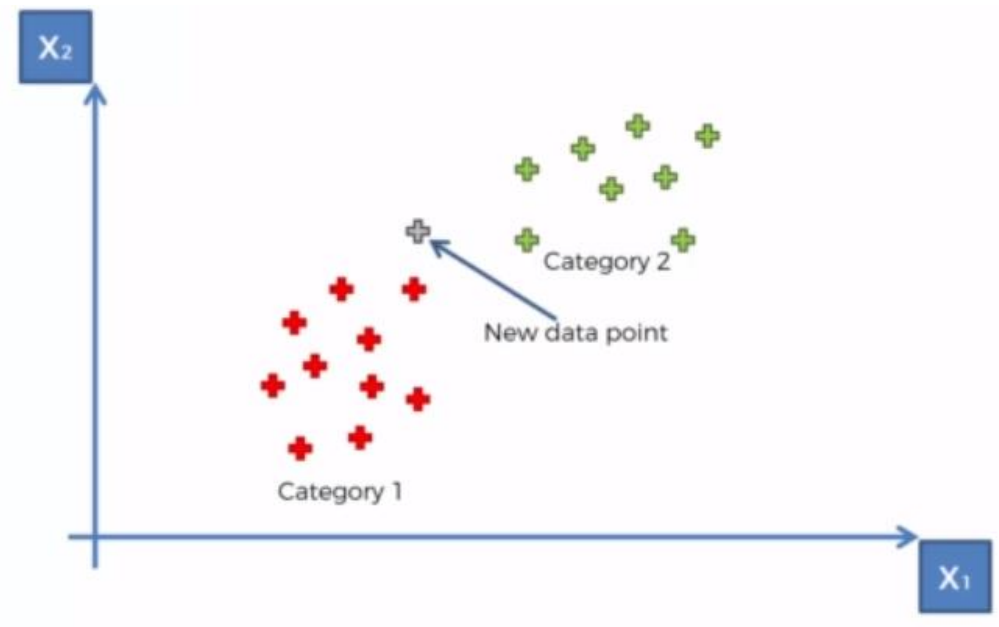


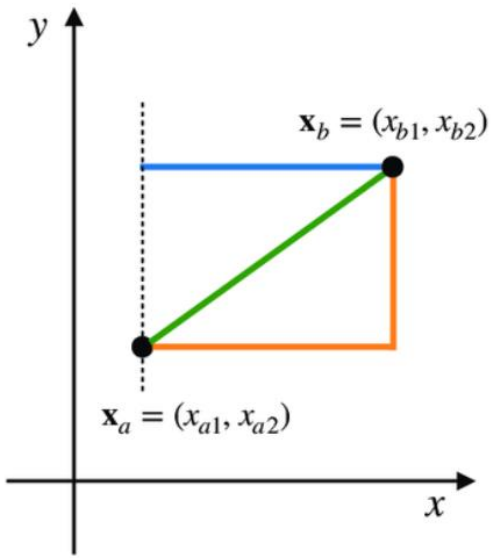
Consider that you have two category. If the new data point is included, will it fall under category 1 or 2?

After executing the KNN algorithm, we will be able to determine which category the new data point falls under.

Classification – KNN

Step 1





$p = 2$ Euclidean distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_2 = (|x_{a1} - x_{b1}|^2 + |x_{a2} - x_{b2}|^2)^{\frac{1}{2}}$$

$p = 1$ Manhattan distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_M = |x_{a1} - x_{b1}| + |x_{a2} - x_{b2}|$$

$p = \infty$ Chebyshev distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_\infty = \max\{|x_{a1} - x_{b1}|, |x_{a2} - x_{b2}|\}$$

The formula for Minkowski distance is as follows:

D(x, y) = (∑|xi - yi|^p)^(1/p)

Calculate the Euclidian/Manhattan/minkowski distance between new data point and other data points. If K=5, then choose only 5 points which are closer to new data point

Classification – KNN

Points A(2, 3) and B(5, 1)

Manhattan distance = $|2-5| + |3-1| = 3 + 2 = 5$

Points A(2, 3) and B(5, 1)

Euclidean distance = $\sqrt{((5-2)^2 + (1-3)^2)} = \sqrt{(9 + 4)} = \sqrt{13}$

Points A(2, 3) and B(5, 1)

Chebyshev distance = $\max(|5-2|, |1-3|) = \max(3, 2) = 3$

Classification – KNN

Step 2

1. Number of data points in Category 1= 3
2. Number of data points in Category 2= 2

Category 1 has highest number of data points

Count the number of data points in each category.

Classification – KNN

Step 3

Include the new data point to Category 1 which has highest number of data points

Classification – KNN - Hyperparameters

- **K value**: The number of nearest neighbors to consider.
- **Distance Metric**: How to measure distance between data points (e.g., Euclidean, Manhattan).
- **Weights**: How to weight the neighbors (uniform or distance-based).
- The choice of hyperparameters in KNN, such as the number of neighbors (K value), the distance metric, and the weighting method, can significantly impact the model's performance.

Classification – KNN - Hyperparameters

- **K Value**
- **Small K values** may capture noise in the data.
- **Large K values** provide more generalization but may overlook small patterns.
- Use **cross-validation** to find the optimal K value.
- **Distance Metrics**
- **Euclidean Distance**: Most common, calculated as the straight line distance.
- **Manhattan Distance**: Sum of absolute differences between points.
- **Minkowski Distance:** Generalization of Euclidean and Manhattan distances.

Classification – KNN – Feature Scaling

- **Importance**: Distance metrics can be affected by the scale of features.
- **Methods**:
  - **Standardization**: Scaling features to have a mean of 0 and a standard deviation of 1.
  - **Normalization**: Scaling features to a range between 0 and 1.
- Feature scaling is crucial in KNN as it ensures that all features contribute equally to the distance metric. Standardization and normalization are common techniques to achieve this.

Build, Train, Predict and Evaluate the K-Nearest Neighbour(KNN) Model– Case Study

Classification – KNN - Python

- **Load Dataset**: **BreastCancer.csv**
- **Preprocess Data**: Handle missing values, encode categorical variables.
- **Split Data**: Train and Test sets.
- **Build Model**: Using **KNeighborsClassifier** from **sklearn**.
- **Train Model**: Fit the model on the training set.
- **Predict**: Make predictions on the test set.
- **Evaluate**: Using **confusion matrix** and accuracy metrics.

Similar for all the models

Classification – KNN - Python

Training the K-NN model on the Training set

```
[ ] from sklearn.neighbors import KNeighborsClassifier
    classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
    classifier.fit(X_train, y_train)
```

K=5

Attaching the classifier to the train dataset

Configuring for KNN model by mentioning the classifier is KNN

Classification – KNN - Python

Making the Confusion Matrix

```
[ ] from sklearn.metrics import confusion_matrix, accuracy_score
    y_pred = classifier.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    print(cm)
    accuracy_score(y_test, y_pred)
```

```
[[103   4]
 [  5  59]]
0.9473684210526315
```

False Positive(FP)

False Negative(FN)

Accuracy Rate (94%)

Number of Errors = FP + FN = 4+ 5 = 9

Classification – KNN - Python

Predicting the value

```
[38] print(classifier.predict(sc.transform([[1173347,1,1,1,1,2,5,1,1,1]])))
```

```
[2]
```

It predicted correctly that class=2. So, the model is working fine.

```
print(X_test)
```

```
[[1173347       1       1 ...       1       1       1]
 [1156017       3       1 ...       2       1       1]
 [ 706426       5       5 ...       4       3       1]
```

```
print(y_test)
```

```
[2 2 4 4 2 2 2 4 2 2 4 2 4 2 2 2 4 4 4 2 2 2 4 2 4 4 2 2 2 4 2 4 4 2 2 2 4
 4 2 4 2 2 2 2 2 2 2 4 2 2 4 2 4 2 2 2 4 2 2 4 2 2 2 2 2 2 2 2 4 4 2 2 2 2
 2 2 4 2 2 2 4 2 4 2 2 4 2 2 4 2 4 2 4 4 4 2 4 4 4 2 2 2 4 4 2 2 4 4 2 2 4
 2 2 4 2 2 2 4 2 2 2 4 2 2 4 4 2 4 2 4 2 2 4 2 2 4 2 4 2 2 2 4 2 2 2 2 4 2
 4 2 4 4 2 2 2 2 4 4 2 4 4 4 4 4 4 2 2 2 2 2 2]
```

Check in BreastCancer.csv file if data is not fully visible

Higher Colleges of Technology

كليات التقنية العليا

Thank You